arXiv:2312.01602v3 [quant-ph] 6 Jun 2025

Quantum Time Series Similarity Measures and Quantum Temporal Kernels

Vanio Markov[1], Vladimir Rastunkov[2], and Daniel Fry[2]

[1]Wells Fargo
[2]IBM Quantum, IBM Research

June 10, 2025

Abstract

This article presents a quantum computing approach to designing of similarity measures and kernels for classification of stochastic symbolic time series. In the area of machine learning, kernels are important components of various similarity-based classification, clustering, and regression algorithms. An effective strategy for devising problem-specific kernels is leveraging existing generative models of the example space. In this study we assume that a quantum generative model, known as quantum hidden Markov model (QHMM), describes the underlying distributions of the examples. The sequence structure and probability are determined by transitions within model's density operator space. Consequently, the QHMM defines a mapping from the example space into the broader quantum space of density operators. Sequence similarity is evaluated using divergence measures such as trace and Bures distances between quantum states. We conducted extensive simulations to explore the relationship between the distribution of kernel-estimated similarity and the dimensionality of the QHMMs Hilbert space. As anticipated, a higher dimension of the Hilbert space corresponds to greater sequence distances and a more distinct separation of the examples. To empirically evaluate the performance of the kernels, we defined classification tasks based on a simplified generative model of directional price movement in the stock market. We implemented two widely-used kernel-based algorithms — support vector machines and k-nearest neighbors — using both classical and quantum kernels. Across all classification task scenarios, the quantum kernels consistently demonstrated superior performance compared to their classical counterparts.

1 Introduction

Many machine learning algorithms for time series use various measures of similarity between examples to solve a wide range of problems, including classification, regression, density estimation, and clustering. The measures of similarity play a critical role in the k-NN algorithms, feature-based algorithms, and kernel methods. Various measures are intended to capture different types of similarity within time series data. One category of similarities is estimated directly within the original time series space. In cases where precise timing and synchronization of data points are essential, time series are compared synchronously, in lock-step, using an one-to-one alignment of corresponding data points. Some common lock-step time series similarity measures include Euclidean distance, linear time warping (LTW)[1], Pearson correlation coefficient [2], Spearman rank correlation [3]. The synchronous methods are not applicable for series with different lengths and cannot capture the local dependencies among adjacent positions within time series data. These difficulties can be resolved by applying non-linear mappings between the time series which allow comparison of one to many points. This approach provides a degree of flexibility or "warping" when aligning two time series. Such flexibility makes it possible to find optimal alignment between time series that may exhibit variations in timing, speed, or phase. Examples of these elastic similarity measures include the longest common sub-sequence (LCSS) [4], dynamic time warping (DTW) [5], edit distance [6], move-split-merge (MSM) distance [7], piecewise linear approximation (PLA) distance [8], symbolic aggregate approximation (SAX) distance [9], elastic ensemble distance (EED) [10]. The synchronous and elastic similarities are estimated over the entire time series and are often referred to as "global similarities". Alternatively, some approaches measure

similarity with respect to local but representative patterns known as "shapelets" [11] within time series. For each time series, the similarity between the time series and each shapelet in a "dictionary" is calculated, resulting in distance features. Distance features may also be defined by global similarity measures. Any similarity measure is associated with a "dissimilarity" feature space, where each time series is represented by its distance vector to the other time series.

Another group of highly successful, similarity-based methods, known as *kernel methods* [12] utilize a function that estimates the similarity between examples by computing the dot product of their images in a higher-dimensional feature space. When the kernel functions satisfy Mercer's theorem [13, 14] they can calculate the dot product directly in the example space, without explicitly construction of feature vectors [15]. The objective of this method is to ensure that the high-dimensional feature representations of the examples are linearly separable with respect to their class labels [14]. Kernel methods enable the construction of linear discriminative algorithms for learning domains with non-linear decision boundaries. Examples of such algorithms include support vector machines [15], kernel logistic and ridge regression [16], and kernel discriminant analysis [17], among others. Kernel methods are also used in unsupervised learning algorithms to capture non-linear structures in data. These include kernel principal component analysis [18, 19, 20], non-linear independent component analysis [21, 22], kernel K-means clustering [23], kernel spectral clustering [24], and kernel density estimation [25].

Most general kernel functions, such as polynomial functions, Gaussian radial basis functions, sigmoid kernels, and hyperbolic tangent functions, rely on simple pairwise distances, such as the Euclidean distance, between data points in the example space. However, these kernels may not be suitable when the example space contains time series with varying lengths. One direct solution to this problem is to utilize elastic similarity measures as demonstrated in the case of *DTW kernels* [26, 27]. If the goal is to estimate significant local similarities the method of local alignment score [28] can be used to define the kernel. This approach has been combined in [29] with a convolution kernel [30] to derive a *local alignment kernel*. Finally, for time series classification tasks involving symbolic, noisy, or high-dimensional data, it is appropriate to use a kernel based on the symbolic aggregate approximation (SAX) similarity measure [9].

The selection of kernel type for a specific machine learning task depends on the nature of the task and the specific characteristics of the process under consideration. It is important to establish a systematic method to define the kernel class and parameters for any particular learning scenario.

One approach to kernel design involves leveraging information from an existing, unsupervised, generative model of the examples domain. Such models define underlying distributions within the examples domain. The underlying distributions define feature spaces where the examples are represented. There are multiple ways for implementing of generative model-based kernels. The *P-kernels* [23, 31] estimate the similarity between two time series based on the joint probability of their inference state sequences, also known as inference paths. These kernels assume the existence of a probabilistic generative model, such as a hidden Markov model or Gaussian mixture model. For a time series, the model defines probabilities of all its inference paths — sequences of model states or mixture components. This distribution serves as the kernel's feature space. In this approach, each time series is transformed into a feature vector of probabilities over all possible inference paths. The similarity between two series is then computed by taking the dot product of their respective feature vectors, with each component weighted by the probability of the corresponding inference path. Because the feature vector contains probabilities for all possible inference paths, these kernels are also referred to as *marginalisation kernels*.

In another generative approach, known as *probability product kernel* [32], each time series is mapped to a distribution within a parameterized class of distributions. The probabilistic models for the individual time series are identified using maximum likelihood estimation. The probability product kernel is defined as standard inner product between densities. Since the vectors in the feature space are probability distributions, the distance between them can also be estimated using common divergence measures like the Kullback-Leibler divergence, Jensen-Shannon divergence, or other statistical distance measures [33, 34]. It is important to note, that these kernel functions usually do not satisfy the Mercer's theorem, which can lead to various issues in kernel-based learning algorithms.

One of the first methods for exploiting the knowledge captured by a probabilistic model to define a similarity metrics is to embed the time series in the gradient space of the generative model. This approach is known as *Fisher kernel* [35]. The gradient vectors of a series log-likelihood with respect

to a model parameters describe how that parameters contribute to the generative process. These parameters can encompass transition and emission probabilities in the case of hidden Markov models, expectations, and variances in the context of Gaussian mixtures, and more. The gradient vector is known as *Fisher score* [36]. If two series have similar gradient vectors, it indicates the model generates them in a similar manner. Consequently, we consider the series being close to each other in terms of their generative processes. The Fisher kernel is defined as the inner product of the gradient vectors of any two time series.

In this article, we investigate similarity measures and their associated kernels for symbolic time series sampled from an underlying stochastic process language. We assume the existence of a Markovian quantum generative model of the language [37], which defines two classes of probability distributions: one over the observed sequences and the other over their inference paths of quantum states represented by quantum density operators. The feature spaces of our kernels are the spaces of quantum density operators. We estimate similarity in these spaces using divergence measures such as trace and Bures distances.

The article's organization is as follows: **section 2** provides a brief introduction to concepts, definitions, and features related to stochastic process languages and the corresponding generative models. We formalize classification problems for symbolic time series based on the concept of a classification map. Depending on the domain of the classification map, we define two types of classification tasks: structural and predictive. We discuss the definitions and requirements of the kernel functions utilized in classification problems. In **section 3** we define a quantum generative model — quantum hidden Markov model (QHMM) — which serves as the foundation of our study. The model is defined as a completely positive trace-preserving (CPTP) map, enabling comprehensive definitions of symbolic sequence probabilities in the example space and generative state sequences in quantum density space. Subsequently, we discuss an unitary physical implementation of the model introduced in [37]. In **section 4**, we formally specify quantum predictive and structural kernels based on the introduced QHMM. We explain the intuition behind these kernels and establish some important probabilistic properties. **Section 5** presents the conducted simulations and empirical investigations regarding the impact of the QHMM's Hilbert space dimension and the distance distributions in the context of classification tasks. Here, we empirically compare the performance of the introduced quantum kernels with their classical counterparts. The discussion of the physical implementation of the proposed kernels in the quantum circuits computing model is presented in **section 6**. The contributions of the article, its potential impact on the field, and the future research directions are presented in **section 7**.

2 Preliminaries

2.1 Stochastic Process Languages

We consider a class of observable, discrete-time stationary stochastic processes denoted by

$$\{y_t : t \in \mathbb{N}, y_t \in \Sigma\}, \tag{1}$$

where $\Sigma = \{a_1, \ldots, a_m\}$ is a finite set of symbols, called an *alphabet*.

The set of all finite sequences over the alphabet Σ, including the empty sequence ϵ, is denoted by Σ^*. The set of all sequences with length exactly t is denoted by Σ^t. Any subset of Σ^* is a *language* L over the alphabet. The sequences belonging to a language are referred to as *words*.

The set of sequences resulting from observations or measurements of the evolution of a discrete-time process is called a *process language*. In this context, the index t is non-negative and is interpreted as *time* with the process sequences being considered as *time series*. It is straightforward to verify that if a word results from the observation of a process, then every one of its subwords has also been observed. Therefore, the process languages are *subword-closed*.

A *stochastic* process language L is defined as a process language along with a set

$$D^L = \left\{D_t^L : t \geq 0\right\} \tag{2}$$

of finite dimensional probability distributions D_t^L, each of which is defined on the sequences with length exactly t:

$$D_t^L = \left\{P[\mathbf{y}] : \mathbf{y} \in \Sigma^t, \sum_{\mathbf{y} \in \Sigma^t} P[\mathbf{y}] = 1\right\} \tag{3}$$

2.2 Probabilistic Generative Models

We will consider probabilistic generative models for the stochastic process languages. These models describe the underlying distribution of the language assigning probability to each sequence from Σ^*. Examples of such models are variational autoencoders (VAEs), Gaussian mixture models (GMMs), hidden Markov models (HMMs), latent Dirichlet allocation (LDA), Bayesian networks, Boltzmann machines, etc [38].

A generative model of stochastic language can be defined assuming that each observation y_t depends on the state of a non-observable or *hidden* finite-state process

$$\{x_t : t \in \mathbb{N}, x_t \in S\}, \tag{4}$$

where $S = \{s_1 \dots s_n\}$ are the process states.
The joint process

$$\{y_t, x_t : t \in \mathbb{N},\, y_t \in \Sigma,\, x_t \in S\} \tag{5}$$

is assumed to be stationary and is described by a linear model

$$\begin{aligned} x_{t+1} &= Ax_t \\ y_t &= Bx_t, \end{aligned} \tag{6}$$

where A is a row-stochastic transition matrix and B is column-stochastic emission matrix.

At any moment in time t, the model is in a superposition (a stochastic mixture) of its hidden states, described by a stochastic vector $x_t \in \mathbb{R}^n$. The state component x_t^i represents the probability of the process being in state $x_t = s_i$. The distribution of observable symbols at time t is defined by the stochastic vector $y_t \in \mathbb{R}^m$ as the probability to observe $y_t = a_i$ is denoted by y_t^i. This model is known as a (classical) hidden Markov model (HMM).

For every HMM $\mathbf{M}$ we can define a set of *observable operators* $\mathbf{T}$ as follows

$$\mathbf{T} = \{T_a : T_a = AB_a, a \in \Sigma\}, \tag{7}$$

where $B_a = \mathrm{diag}(B[a,i], i \in [1.n])$ are diagonal matrices and $B[a,i]$ represents the emission probability of observing symbol a in state s_i [39, 40].

Every component $T_a[i,j]$ of an observable operator T_a defines the conditional probability to observe symbol a when the process state evolves from state s_i to s_j.

$$T_a[i,j] = P[s_j \mid s_i, a].$$

For any sequence $\mathbf{a} = a_1 \dots a_t$ we also can define an observable operator as follows:

$$T_{\mathbf{a}} = T_{a_l} \dots T_{a_1}.$$

A HMM $\mathbf{M}$ defines probability of every observable sequence $\mathbf{a} = a_1 \dots a_t$ as:

$$P[\mathbf{a}|\mathbf{M}] = \mathbf{1}T_{a_t} \dots T_{a_1}\mathbf{x}_0 = \mathbf{1}T_{\mathbf{a}}\mathbf{x}_0, \tag{8}$$

where $\mathbf{1}$ is the unit row vector with dimension n, and $\mathbf{x}_0$ is the *initial* state distribution of the model. With every model $\mathbf{M}$ we associate a *sequence function* $f^{\mathbf{M}} : \Sigma^* \to [0,1]$ defined as:

$$f^{\mathbf{M}}(\mathbf{a}) = P(\mathbf{a}|\mathbf{M}), \forall \mathbf{a} \in \Sigma^*. \tag{9}$$

Through the function $f^{\mathbf{M}}$ the model defines a stochastic process language L_M (3):

$$D_t^{\mathbf{M}} = \{f^{\mathbf{M}}(\mathbf{a}) : \mathbf{a} \in \Sigma^t\} \tag{10}$$

2.3 Classification Problem for Stochastic Languages

We consider the *symbolic time series classification problem* for a stochastic language L and a finite set of class labels $C = \{c_1, \dots, c_k\}$. It is assumed that there exists an unknown *probabilistic classification map* $\mathcal{P} : L \to P[C]$, which assigns to each sequence $\mathbf{y} \in L$ a probability distribution over the class labels.

We distinguish between two types of probabilistic classification maps:

Structural classification maps. These maps, which we call *structural*, assign class probabilities based solely on the structure of the observed sequence:

$$p_s(\mathbf{y}, c) = P[c \mid \mathbf{y}], \quad \mathbf{y} \in L,\ c \in C. \tag{11}$$

In structural classification, the class of a sequence depends only on its observed pattern. Examples include protein family classification based on amino acid sequences [41], detection of anomalous financial transactions [42], and intrusion detection in cybersecurity [43].

Predictive classification maps. The second type of classification maps assign class probabilities depending on the expected *future evolution* of the observed sequence. For a fixed horizon $k > 0$, the *predictive classification map* is defined as:

$$p_p(\mathbf{y}, c) = \sum_{\mathbf{z} \in \Sigma^k} P[\mathbf{z} \mid \mathbf{y}] \cdot P[c \mid \mathbf{yz}], \quad \mathbf{yz} \in L,\ c \in C, \tag{12}$$

where $P[\mathbf{z} \mid \mathbf{y}]$ denotes the probability of observing continuation $\mathbf{z}$ of length k after observing $\mathbf{y}$, and $P[c \mid \mathbf{yz}]$ is the class probability conditioned on $\mathbf{yz}$.

We refer to classification problems defined by (12) as *predictive*. These problems arise in domains where future dynamics influence class membership, such as weather forecasting [44], stock market trend prediction [45], patient health monitoring [46], energy load forecasting [47], and traffic flow analysis [48].

Training and learning. A training dataset consists of labeled sequence samples:

$$S = \{(\mathbf{y}_i, c_i) \mid \mathbf{y}_i \in Y,\ c_i \in C,\ i \in [1, \ell]\},$$

where $Y = \left\{\mathbf{y}_i \sim D_i^L\right\}$ is a set of finite sequences sampled from respective distributions D_i^L over L. The goal of the classification problem is to learn an approximation $\widehat{\mathcal{P}}$ to the true classification map $\mathcal{P}$.

Evaluation. The quality of the learned classifier $\hat{p}(\mathbf{y}, c)$ is typically evaluated using probabilistic scoring rules such as the Brier score, negative log-likelihood, or proper scoring metrics that assess the predictive distribution over labels. These measures quantify the fidelity of the learned stochastic classification behavior.

2.4 Kernel Functions

A common approach to solving classification problems is to establish a similarity measure for pairs of examples that is consistent with their respective classes: examples within the same class should be considered closer according to this measure compared to examples from different classes. The similarity measure can be formalized as a *kernel function*, which computes the similarity or distance between pairs of examples in the input domain. We will assume that the example domain is a stochastic process language L as discussed in Section 2.1. A kernel function is formally defined as follows:

$$K : L \times L \to \mathbb{R}, \tag{13}$$

where K is a symmetric, positive semi-definite function.
The idea behind this definition is to map the examples into a high-dimensional feature space where the data becomes more separable or even linearly separable, and then to compute similarity within that space. Let's consider an implicit or explicit map of the input examples to a high-dimensional Hilbert space H:

$$\phi : L \to H. \tag{14}$$

If κ is a symmetric positive semi-definite measure in the Hilbert space:

$$\kappa : H \times H \to \mathbb{R}, \tag{15}$$

then the kernel function is specified as:

$$K(x_1, x_2) = \kappa(\phi(x_1), \phi(x_2)) \tag{16}$$

We construct a kernel function using the state-space of the stochastic generative model (6). The model assigns probability to every observable sequence by equation (8). The model also defines a mapping between the examples $\mathbf{y} = y_1 \dots y_t$ and the vectors of probabilities of observing an example $\mathbf{y}$ from each of the n possible states or inference paths:

$$\phi_{\mathbf{M}}(\mathbf{y}) = T_{y_t} \dots T_{y_1} \mathbf{x}_0 = T_{\mathbf{y}} \mathbf{x}_0 \tag{17}$$

construction of classification and regression learning algorithms that are nonlinear in the input space L while having linear counterparts within the Hilbert space H.

In the next session we assume the existence of a quantum generative model for the language L and apply this approach in the Hilbert space of the model.

3 Quantum Hidden Markov Models

We will consider stochastic process languages defined by quantized HMMs known as quantum hidden Markov models (QHMMs) [49]. A QHMM is a complete positive quantum operation [50] or quantum channel, describing how a composite quantum system evolves according to its internal dynamics and simultaneously parts of it are observed by measurement. The model combines unitary hidden states evolution with the emission of observations correlated with the hidden states. The QHMM is a *quantum stochastic generator*, formally defined as follows:

Definition 1 (Quantum hidden Markov model [49])**.** *A quantum HMM (QHMM)* $\mathbf{Q}$ *over an* N*-dimensional Hilbert space* $\mathcal{H}$ *is a 4-tuple:*

$$\mathbf{Q} = \{\Sigma,\, \mathcal{H},\, \mathcal{T} = \{T_a\}_{a\in\Sigma},\, \rho_0\}, \tag{18}$$

where

- Σ *is a finite alphabet of observable symbols.*
- $\mathcal{H}$ *is an* N*-dimensional Hilbert space. It defines the state-space of the model* $\mathbf{Q}$ *as the set of associated density operators* $D(\mathcal{H})$*.*
- $\mathcal{T}$ *is a CPTP map (quantum channel)* $\mathcal{T} : D(\mathcal{H}) \to D(\mathcal{H})$*.*
- $\{T_a\}_{a\in\Sigma}, \sum_{a\in\Sigma} T_a^\dagger T_a = I_N$ *is an operator-sum representation of* $\mathcal{T}$ *in terms of a complete set of Kraus operators* T_a*.*
- ρ_0 *is an initial state,* $\rho_0 \in D(\mathcal{H})$*, where* $D(\mathcal{H})$ *is the space of density operators defined in* $\mathcal{H}$*.*

The space $D(\mathcal{H})$ is a convex set, representing statistical ensembles of quantum states, which consists of all positive semi-definite operators ρ on $\mathcal{H}$ with $\mathrm{tr}(\rho) = 1$. When the map $\mathcal{T} = \{T_a\}_{a\in\Sigma}$ is applied to a state $\rho \in D(\mathcal{H})$, the model $\mathbf{Q}$ defines measurement outcome $a \in \Sigma$ with probability:

$$P\big[a|\rho\big] = \mathrm{tr}(T_a \rho) \tag{19}$$

and the system's state after the measurement becomes:

$$\rho_a = \frac{T_a \rho}{P\big[a|\rho\big]}. \tag{20}$$

The completeness of the set of Kraus operators $\{T_a\}_{a\in\Sigma}$ guarantees that the measurement probabilities in each state define a distribution:

$$D_\rho^{\mathbf{Q}} = \{P\big[a|\rho\big] : a \in \Sigma\}. \tag{21}$$

If the operation $\mathcal{T}$ is applied t times starting at the initial state a sequence $\mathbf{y} = y_1 \dots y_t$ will be observed with probability

$$P\big[\mathbf{y}|\rho_0\big] = \mathrm{tr}(T_{\mathbf{y}} \rho_0). \tag{22}$$

and the final state will be

$$\rho_{\mathbf{y}} = \frac{T_{\mathbf{y}} \rho_0}{P\big[\mathbf{y}|\rho_0\big]}, \tag{23}$$

where

$$T_{\mathbf{y}} = T_{y_t} \dots T_{y_1}$$

For each QHMM $\mathbf{Q}$ the equation (22) defines a *sequence function* $f^{\mathbf{Q}}$ as follows:

$$f^{\mathbf{Q}}(\mathbf{y}) = P\big[\mathbf{y}|\rho_0\big], \forall \mathbf{y} \in \Sigma^*. \tag{24}$$

The sequence function (24) defines a distribution $D_t^{\mathbf{Q}}$ over the sequences of length t for $\forall t > 0$, since the quantum operation $\mathcal{T}$ is completely positive:

$$D_t^{\mathbf{Q}} = \big\{ f^{\mathbf{Q}}(\mathbf{y}) : \mathbf{y} \in \Sigma^t \big\} \tag{25}$$

Therefore every QHMM $\mathbf{Q}$ defines a *stochastic process language* $L^{\mathbf{Q}}$ (2) over the set of finite sequences Σ^*.

The Definition 1 of a QHMM is based on the concept of a POVM operation acting on a quantum state. The emission of the observable symbols is encoded in the operational elements (Kraus operators) of the quantum operation. This framework provides a convenient way to view QHMMs as channels for quantum information processing. It allows for the analysis of their informational complexity, expressive capacity, and establishes connections to stochastic process languages and the corresponding automata. However, this approach cannot be directly used for implementation of the QHMMs on quantum computing hardware. A critical result in quantum information theory — the Stinespring's representation theorem [51] — provides approach to the physical implementation of quantum channels and correspondingly of QHMMs. According to the theorem any quantum channel can be realized as a unitary transformation on a larger system (the combined hidden state and observable systems) followed by a partial trace operation that discards the observable system. An immediate consequence of this result is the following definition of QHMMs in the unitary circuits model of computation:

Definition 2 (Unitary quantum hidden Markov model [37])**.** *A unitary quantum HMM* $\mathbf{Q}$ *over a finite alphabet of observable symbols* Σ *and finite* N*-dimensional Hilbert space is a 6-tuple:*

$$\mathbf{Q} = \big\{ \Sigma, \mathcal{H}_S, \mathcal{H}_E, U, \mathcal{M}, R_0 \big\} \tag{26}$$

where

- Σ *is a finite set of* m *observable symbols.*
- $\mathcal{H}_S$ *is the Hilbert space of the hidden state system of dimension* N*.*
- $\mathcal{H}_E$ *is the Hilbert space of an auxiliary emission system with dimension* $m \leq M \leq N^2$ *and orthonormal basis* $E = \{|e_i\rangle\}_{i=0}^{M-1}$*.*
- U *is a unitary operator defined on the bipartite Hilbert space* $\mathcal{H}_S \otimes \mathcal{H}_E$*.*
- $\mathcal{M}$ *is a bijective map* $\mathcal{P}_m^E \to \Sigma$*, where* $\mathcal{P}_m^E$ *is an* m*-element partition of* E*.*
- $R_0 = \rho_0 \otimes |e_0\rangle \langle e_0|$ *is an initial state.*

An implementation of QHMM where $\rho_0 = |s_0\rangle \langle s_0|$ is presented on Figure 1. This unitary physical realization of a quantum channel (*Definition* 1) follows from the *operator-sum representation* [50] of the quantum operation $\mathcal{T}$:

$$\mathcal{T}\cdot = \sum_{e \in \{|e_i\rangle\}_{i=0}^{M-1}} T_e \cdot T_e^\dagger. \tag{27}$$

where the Kraus operators $\{T_e\}$ act on the hidden system states and are defined as follows:

$$T_e = \big(I^N \otimes \langle e|\big)\, U \,\big(I^N \otimes |e_0\rangle\big)\,. \tag{28}$$

The Kraus operators depend on the unitary U, and the arbitrarily selected orthonormal basis $\{|e\rangle\}$ of the emission system.

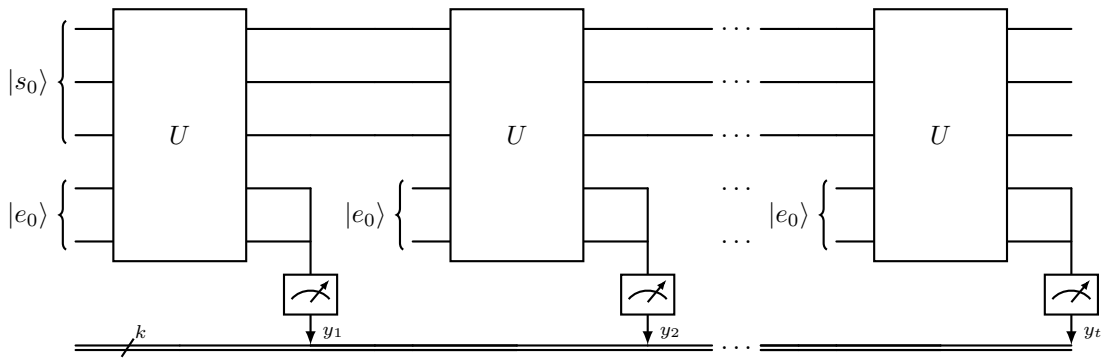

Figure 1: Implementation of QHMM.

State	Direction Bias	0	1	2	3	$P[0\|State]$	$P[1\|State]$
0	Bear - Tendency Down	0.50	0.10	0.15	0.25	0.80	0.20
1	Bull - Tendency Up	0.10	0.50	0.25	0.15	0.20	0.80
2	Transition to Bear	0.25	0.15	0.50	0.10	0.40	0.60
3	Transition to Bull	0.15	0.25	0.10	0.50	0.60	0.40

Table 1: Hidden states descriptions, transition probabilities, and observation probabilities.

Example 1. In [37] we discussed a stochastic process language generated by a simple classic hidden Markov model (HMM) of directional market price movements. The market is assumed to have four hidden states and observable symbols $0, 1$ corresponding to price move down and up. Table 1 provides the transition and emission probabilities of the model. The hidden states transition graph and the distributions defining the stochastic process language are presented on Figure 2. In [37] we proved that every classical HMM of n states can be simulated by a QHMM (26) in Hilbert space with dimension $\sqrt{n}$. Following this result we identified a generative model — a QHMM with one state qubit and one emission qubit (Figure 3) — which reproduces exactly the distributions of the market process language as shown on Figure 4. The first part of the circuit prepares a *maximally mixed* initial state, the middle part performs the hidden state transition, and at the end is the measurement of the emission subsystem in a learned orthonormal basis.

4 Quantum Generative Kernels

In this section, we introduce sequence similarity measures based on the quantum generative model defined by equations (18) and (25). We assume that a QHMM which defines the distribution set (25) of the example space is specified either as a quantum channel (18) or unitary model (26).

4.1 Predictive Generative Kernels

The predictive similarity measure is defined by the stochastic distance between the expected future evolutions of the sequences. Since the evolution of a Markovian process depends only on its current state, it is easy to verify that if the quantum states ρ_1 and ρ_2 defined by any two sequences $\mathbf{y^1}$ and $\mathbf{y^2}$ (23):

$$\rho_1 = \frac{T_{\mathbf{y^1}}\rho_0}{P\big[\mathbf{y^1}|\rho_0\big]}$$

$$\rho_2 = \frac{T_{\mathbf{y^2}}\rho_0}{P\big[\mathbf{y^2}|\rho_0\big]},$$

are the same, then the sequences $\mathbf{y^1}$ and $\mathbf{y^2}$ are considered *equivalent* in the sense, that they define the same future distributions of the observables.

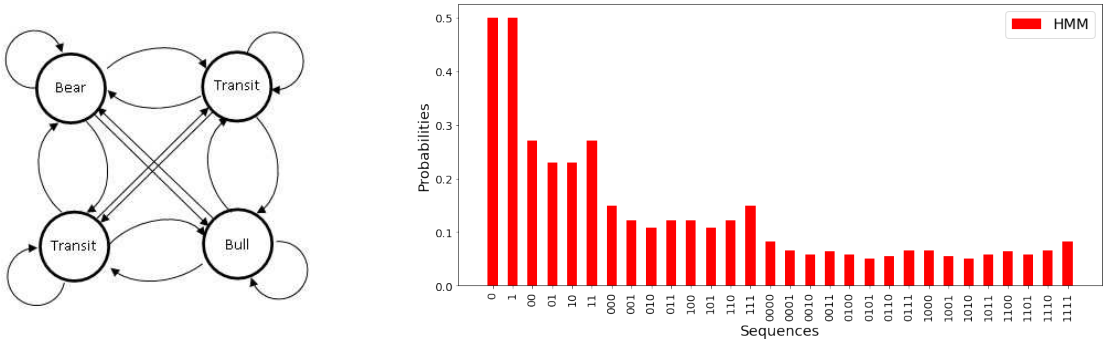

Figure 2: Left: Market hidden states transition graph. Right: Distributions of observed sequences.

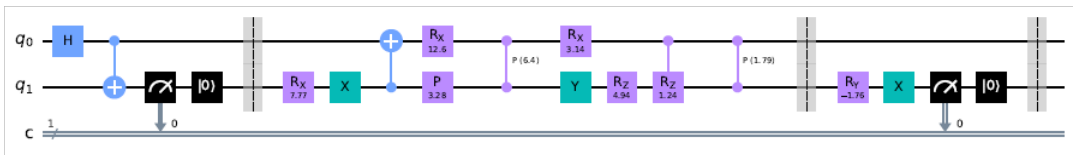

Figure 3: QHMM defining market distribution

The following proposition generalizes this idea by proving that if two states are close in trace norm, the corresponding forward distributions of the sequences are close in total variation distance.

Proposition 1. *For any two states ρ_1 and ρ_2 of a QHMM $\mathbf{Q}$ the total variation distance of the observable distributions $P\big[\mathbf{z}|\rho_1\big]$ and $P\big[\mathbf{z}|\rho_2\big]$ for $\mathbf{z} \in \Sigma^k, k > 0$ is bounded by the trace distance between the states ρ_1 and ρ_2.*

Proof. The similarity between the quantum states ρ_1 and ρ_2 can be estimated by their trace distance:

$$\mathcal{D}(\rho_1, \rho_2) = \frac{1}{2}\mathrm{tr}|\rho_1 - \rho_2|, \tag{29}$$

where

$$|\rho_i| \equiv \sqrt{\rho_i^\dagger \rho_i}, i = 1, 2$$

The probabilities the same sequence $\mathbf{z} \in \Sigma^k$ to be observed at states ρ_1 and ρ_2 respectively are (22)

$$P\big[\mathbf{z}|\rho_1\big] = \mathrm{tr}(T_\mathbf{z}\rho_1), \tag{30}$$

and

$$P\big[\mathbf{z}|\rho_2\big] = \mathrm{tr}(T_\mathbf{z}\rho_2). \tag{31}$$

The difference between these probabilities is defined as follows:

$$P\big[\mathbf{z}|\rho_1\big] - P\big[\mathbf{z}|\rho_2\big] = \mathrm{tr}(T_\mathbf{z}(\rho_1 - \rho_2)). \tag{32}$$

Since the operation T_z is trace non-increasing we have:

$$P\big[\mathbf{z}|\rho_1\big] - P\big[\mathbf{z}|\rho_2\big] \leq \mathrm{tr}(\rho_1 - \rho_2). \tag{33}$$

If we assume that $P\big[\mathbf{z}|\rho_1\big] \geq P\big[\mathbf{z}|\rho_2\big]$ then

$$\big|P\big[\mathbf{z}|\rho_1\big] - P\big[\mathbf{z}|\rho_2\big]\big| \leq 2\mathcal{D}(\rho_1, \rho_2). \tag{34}$$

The total variation distance between the distributions $P_1(\mathbf{z}) = P\big[\mathbf{a}|\rho_1\big]$ and $P_2(\mathbf{z}) = P\big[\mathbf{z}|\rho_2\big]$ is

$$\delta(P_1, P_2) = \sup_{\mathbf{z}\in\Sigma^k} \big|P_1(\mathbf{z}) - P_2(\mathbf{z})\big| \tag{35}$$

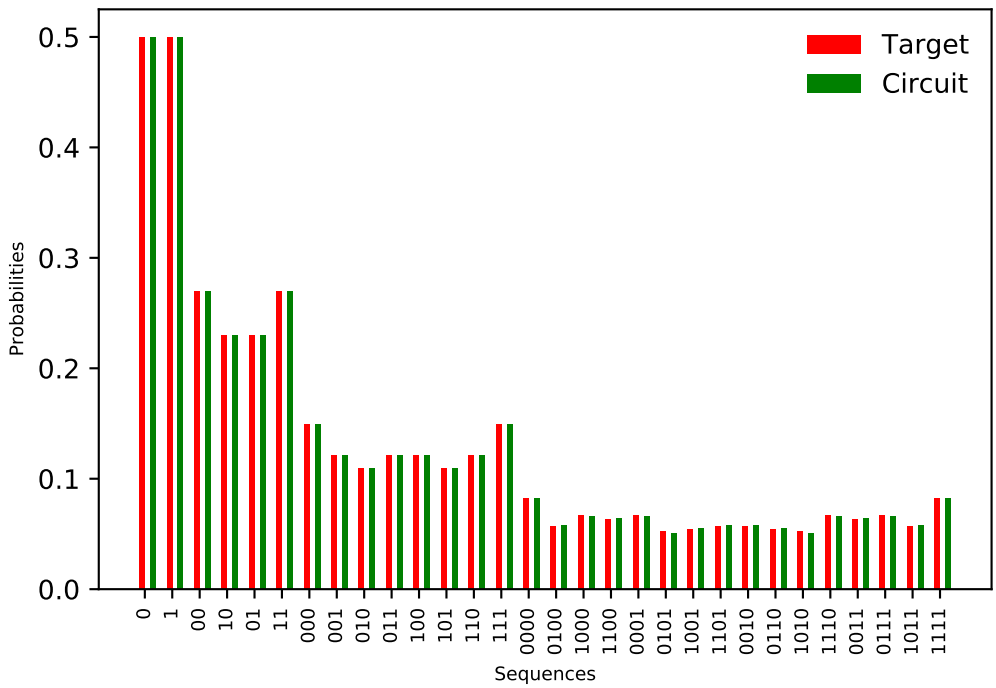

Figure 4: Targeted and Learned Market Process Language Distributions

Since (34) is valid for any $\mathbf{z} \in \Sigma^k$, then from (35) follows:

$$\delta(P_1, P_2) \leq 2\mathcal{D}(\rho_1, \rho_2) \tag{36}$$

□

This proposition motivates us to define the similarity between every sequence pair $\mathbf{y}^1$ and $\mathbf{y}^2$ by mapping them to the quantum states ρ_1 and ρ_2 of the model (18), and calculating the trace distance between these states. Further in this chapter we will proof that this measure of similarity pertains to the future evolution of the time series.

We define the predictive generative kernel as follows:

Definition 3 (Predictive Generative Quantum Kernel)**.** *Let* $\mathbf{Q}$ *be a Quantum Hidden Markov Model (QHMM) with domain of observable sequences* $L^{\mathbf{Q}}$*. For any two sequences* $\mathbf{y}^1, \mathbf{y}^2 \in L^{\mathbf{Q}}$*, the predictive generative quantum kernel* $\kappa_p : L^{\mathbf{Q}} \times L^{\mathbf{Q}} \to \mathbb{R}$ *is defined as:*

$$\kappa_p(\mathbf{y}^1, \mathbf{y}^2) = \exp\left(-\mathcal{D}\left(\phi(\mathbf{y}^1), \phi(\mathbf{y}^2)\right)\right), \tag{37}$$

where $\phi : L^{\mathbf{Q}} \to \mathcal{D}(\mathcal{H})$ *maps each sequence* $\mathbf{y}$ *to a normalized quantum state* $\rho_{\mathbf{y}}$ *(23):*

$$\phi(\mathbf{y}) = \frac{T_{\mathbf{y}}\rho_0}{P[\mathbf{y} \mid \rho_0]}, \tag{38}$$

with $T_{\mathbf{y}}$ *denoting the sequence of Kraus operators associated with* $\mathbf{y}$*,* ρ_0 *the initial state of the system,* $P[\mathbf{y} \mid \rho_0]$ *the probability of observing sequence* $\mathbf{y}$ *given the initial state, and* $\mathcal{D}(\rho_{\mathbf{y}}, \rho_{\mathbf{z}})$ *is the trace distance between the quantum states (29).*

Since the trace distance is a conditionally negative definite (CND) metric on quantum states, it follows from Schoenberg's theorem [14] that the function κ_p (37) is a positive semi-definite (PSD) kernel.

Critical feature of the kernel is that it defines a bound of the dissimilarity of sequences with respect of their probabilistic classification maps (12) as it is demonstrated by the following Proposition.

Proposition 2 (Bound on Predictive Classification Divergence)**.** *Let κ_p be the predictive generative kernel defined by (37). Then for any two sequences $\mathbf{y}^1, \mathbf{y}^2 \in L$ and any horizon $k > 0$, the variation distance between their predictive classification maps is bounded by the negative logarithm of the kernel:*

$$\sup_{c \in C} \left| p_p(\mathbf{y}^1, k, c) - p_p(\mathbf{y}^2, k, c) \right| \leq -C_k \log \left(\kappa_p(\mathbf{y}^1, \mathbf{y}^2) \right). \tag{39}$$

Proof. The variation distance of the class distributions (12) for the sequences is defined as

$$\delta(p_p(\mathbf{y}^1, k, c), p_p(\mathbf{y}^2, k, c)) = \sup_{c \in C} \left| p_p(\mathbf{y}^1, k, c) - p_p(\mathbf{y}^2, k, c) \right| \tag{40}$$

$$\delta(p_p(\mathbf{y}^1, k, c), p_p(\mathbf{y}^2, k, c)) = \sup_{c \in C} \Big| \sum_{\mathbf{z} \in \Sigma^k} (P[\mathbf{z}|\mathbf{y}^1] P[c|\mathbf{y}^1\mathbf{z}]) - \sum_{\mathbf{z} \in \Sigma^k} (P[\mathbf{z}|\mathbf{y}^2] P[c|\mathbf{y}^2\mathbf{z}]) \Big| \tag{41}$$

$$\delta(p_p(\mathbf{y}^1, k, c), p_p(\mathbf{y}^2, k, c)) \leq c^* \Big| \sum_{\mathbf{z} \in \Sigma^k} (P[\mathbf{z}|\mathbf{y}^1]) - P[\mathbf{z}|\mathbf{y}^2]) \Big| \tag{42}$$

$$\delta(p_p(\mathbf{y}^1, k, c), p_p(\mathbf{y}^2, k, c)) \leq c^* \sum_{\mathbf{z} \in \Sigma^k} (\big| P[\mathbf{z}|\mathbf{y}^1]) - P[\mathbf{z}|\mathbf{y}^2] \big|) \tag{43}$$

From Proposition 1

$$\delta(p_p(\mathbf{y}^1, k, c), p_p(\mathbf{y}^2, k, c)) \leq c^* |\Sigma^k| 2\mathcal{D}(\rho_1, \rho_2) \tag{44}$$

$$\delta(p_p(\mathbf{y}^1, k, c), p_p(\mathbf{y}^2, k, c)) \leq -C_k \log(\kappa_p(\mathbf{y}^1, \mathbf{y}^2)), \tag{45}$$

where $c^* \leq 1$ is a constant dependent on the particular probabilistic classification map $P[c|\mathbf{yz}]$ and $C_k = 2c^*|\Sigma^k|$.

□

Several predictive quantum kernels can be defined using alternative distance or similarity measures between quantum states, such as the Bures metric and quantum fidelity.

The Bures metric between two quantum states is defined as:

$$\mathcal{B}(\rho_{\mathbf{y}}, \rho_{\mathbf{z}}) = 2 - 2\sqrt{F(\rho_{\mathbf{y}}, \rho_{\mathbf{z}})},$$

where the quantum fidelity $F(\rho_{\mathbf{y}}, \rho_{\mathbf{z}})$ is defined as:

$$F(\rho_{\mathbf{y}}, \rho_{\mathbf{z}}) = \left(\mathrm{Tr} \left(\sqrt{\sqrt{\rho_{\mathbf{y}}} \rho_{\mathbf{z}} \sqrt{\rho_{\mathbf{y}}}} \right) \right)^2.$$

Using the Bures metric, we can define a valid predictive kernel via negative exponentiation:

$$\kappa_p(\mathbf{y}, \mathbf{z}) = \exp \left(-\mathcal{B}\big(\phi(\mathbf{y}), \phi(\mathbf{z})\big) \right), \quad \mathbf{y}, \mathbf{z} \in L^{\mathbf{Q}}. \tag{46}$$

Since the Bures metric is conditionally negative definite (CND), the kernel defined in (46) is positive semi-definite (PSD) by Schoenberg's theorem.
We can also define a predictive kernel directly from fidelity as follows:

$$\kappa_p(\mathbf{y}, \mathbf{z}) = F(\rho_{\mathbf{y}}, \rho_{\mathbf{z}}). \tag{47}$$

Fidelity is a symmetric, bounded, and positive-definite similarity measure, and thus the function (47) defines a valid PSD kernel.

4.2 Structural Generative Kernels

The classification maps of the structural classification tasks (11) depend on the probabilistic process that generates the sequences. The generative process is defined by the evolution of model's quantum state. For any example $\mathbf{y} \in L^Q, \mathbf{y} = y_1 y_2 \cdots y_n, y_i \in \Sigma, i \in [1, n]$ a *generating state sequence* of $\mathbf{y}$ is defined as (23):

$$\left\{ \rho_i = \frac{T_{y_i} \rho_{i-1}}{tr(T_{y_i} \rho_{i-1})} : i \in [1, n] \right\}. \tag{48}$$

We introduce a kernel that estimates the structural similarity of examples based on the divergence of appropriate statistics derived from the generating state sequences. The structure of each sequence depends on the level of involvement of model's state in the generative process: if a state participates more often, its impact is stronger. To account for this dependency, the structural kernel maps each example to the expectation of its generating states. Then, the structural similarity of sequences is measured by the divergence of the expectations of their generating states. The map of examples to the expectations of quantum states is defined using (48) as follows:

$$\phi_s(\mathbf{y}) = \hat{\rho}_{\mathbf{y}} = \frac{1}{n} \sum_{i \in [1,n]} \rho_i,$$

where $\mathbf{y} = y_1 y_2 \cdots y_n$. It is easy to demonstrate, that the average of a set of density matrices is a density matrix. Therefore, the expectation $\hat{\rho}$ is a quantum state and we can define a valid structural quantum kernel using the trace distance as follows:

$$\kappa_s(\mathbf{y}, \mathbf{z}) = \exp\left(-\mathcal{D}\big(\phi_s(\mathbf{y}), \phi_s(\mathbf{z})\big)\right), \mathbf{y}, \mathbf{z} \in L^Q \tag{49}$$

A valid structural kernel can also be defined using the Bures metric:

$$\kappa_s(\mathbf{y}, \mathbf{z}) = \exp\left(-\mathcal{B}\big(\phi_s(\mathbf{y}), \phi_s(\mathbf{z})\big)\right), \mathbf{y}, \mathbf{z} \in L^Q \tag{50}$$

5 Empirical Evaluation of Quantum Generative Kernels

In this section, we use the stochastic process language discussed in Example 1 and the corresponding generative QHMM (Figure 3) to empirically investigate the behaviour of the introduced kernels in several contexts: dimension of the quantum Hilbert space, type of the classification task, and type of the stochastic divergence measures of the quantum states. The QHMM discussed in Example 1 is minimal, with dimension of the state Hilbert space $N = 2$ (2). To study the impact of the size of the quantum Hilbert space on kernels behaviours we learned and investigated two larger models of the same stochastic process language with dimensions $N = 4$ and $N = 16$ correspondingly. The components of these models: initial state preparation, transition algorithm, and observable emission in circuit quantum computing model are presented in appendix A.

5.1 Quantum Feature Space Dimension Impact

It is important to demonstrate how the introduced kernels support the expectation that mapping the original domain into a high-dimensional Hilbert space enhances the separability of examples. For QHMMs of the Market example with Hilbert space dimensions 2, 4, and 16, we measured the distributions of the distances between examples induced by the kernels. The results are presented in Figures 5 and 6. If a kernel maps a significant number of pairs to short distance ranges, indicating they are not well distinguishable, this behavior could pose a potential problem for kernel learning methods to perform effectively in nonlinear environments. On the other hand, if a kernel assigns significant distances to most pairs of examples, it might be more successful in challenging learning tasks.

The initial observation reveals that the Predictive kernels in low-dimensional Hilbert spaces struggle to distinguish half of the examples while effectively separating the other half. On the other hand, the Structural kernels on 1 and 2 Qubits display better performance by effectively separating a significant portion of the examples. Furthermore, it is evident that the Trace metric yields better separation than the Bures metric. Notably, the 4 Qubit kernel consistently performs well across all scenarios, as the Structural kernel with the Trace metric achieves perfect separation of all pairs of examples. This behavior underscores the direct impact of increasing the dimension of the Hilbert space of the QHMM on the kernel's performance.

5.2 Kernel-Induced Distances with Respect to Classification Tasks

We demonstrate that distances calculated by the structural and predictive kernels are correlated with the classes of corresponding examples. Specifically, examples at shorter distances tend to belong to the same class. We will consider simple structural and predictive classification tasks as defined in

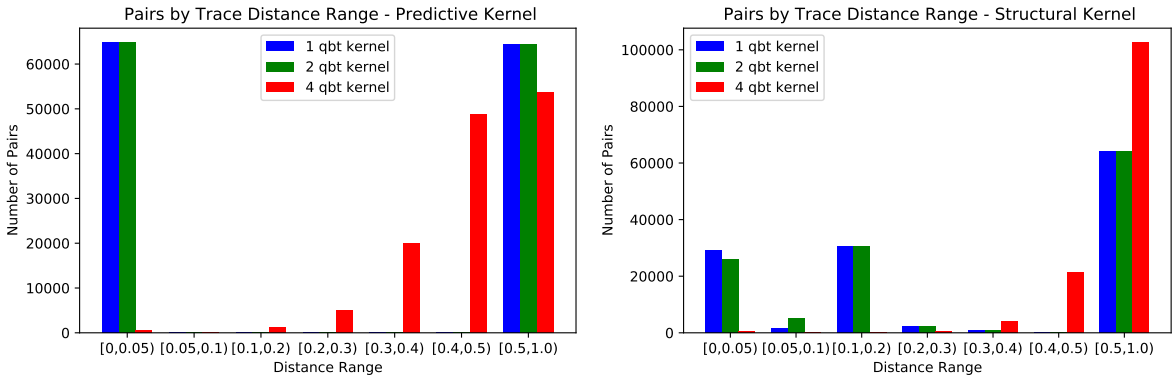

Figure 5: Number of Pairs by Distance - Trace Metric

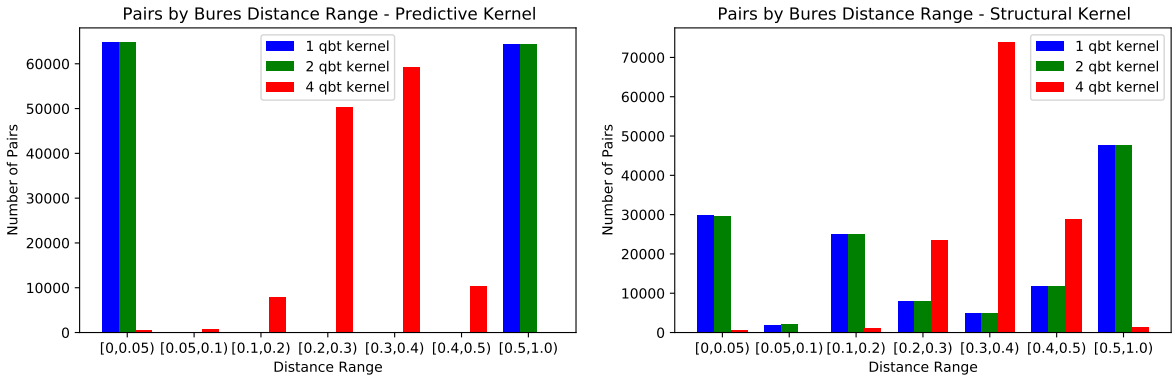

Figure 6: Number of Pairs by Distance - Bures Metric

appendix B. On figures 7 and 8 it is evident that the distances defined by the 4-qubit kernels exhibit an inverse linear relationship with the probability of examples belonging to the same class. The stronger correlation is demonstrated by the 4-qubit predictive kernel in the predictive classification task. Conversely, the performance of the 1-qubit and 2-qubit structural and predictive kernels is the poorest in the structural classification task. Similar results, presented in appendix C, were obtained for the trace metric. Ultimately, the dimensionality of the quantum feature space significantly impacts kernels' performance.

5.3 Quantum Kernels vs Classical Kernels

To compare the performance of the proposed kernels against classical ones, we use the classification tasks described in appendix B. Three common algorithms [52] - random forest classifier, support vector classifier, and kernelized k-nearest neighbours classifier were applied to the classification tasks. The random forest classifier was used as a bench mark and the other were implemented with classical and quantum kernels. As classical kernel was the radial basis function (RBF) kernel defined as follows [14]:

$$K(\mathbf{y}^1, \mathbf{y}^2) = \exp\left(-\frac{\left\|(\mathbf{y}^1 - \mathbf{y}^2)\right\|^2}{2\sigma^2}\right),$$

where $\left\|(\mathbf{y}^1 - \mathbf{y}^2)\right\|$ is the Euclidean distance between the sequences, and σ is a parameter.

In all structural and predictive task scenarios, the quantum kernels exhibited superior performance compared to their classical counterparts and the RFS benchmark. The algorithms were tested on 500 samples split to training and testing at 50% using Trace metric (37) for the kernels. As it can be seen in Table 2 and Table 3 in both classification tasks the quantum kernels outperformed their classical counterparts as the SVC with quantum kernel outperformed the RFS benchmark algorithm as well.

Similar performance were observed with kernels using Bures metric (46), (50) as the results are presented in appendix D.

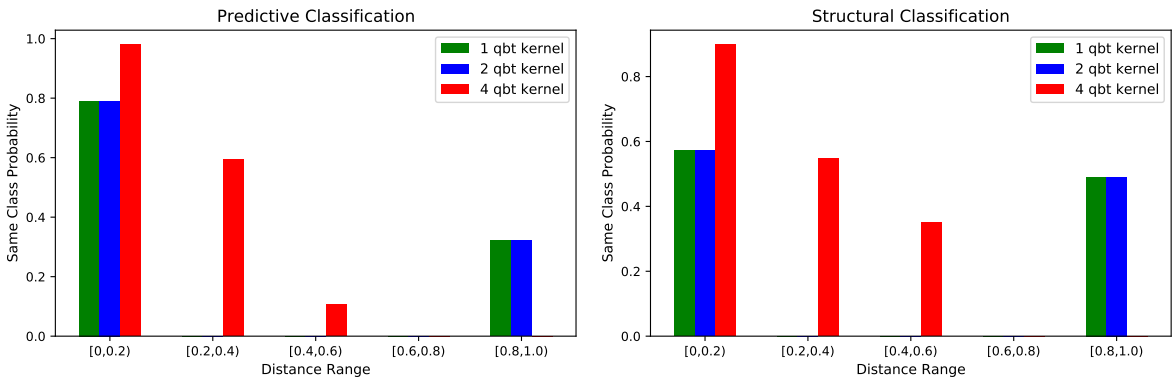

Figure 7: Probability of Examples Belonging to the Same Class - Predictive Kernel, Bures Metric

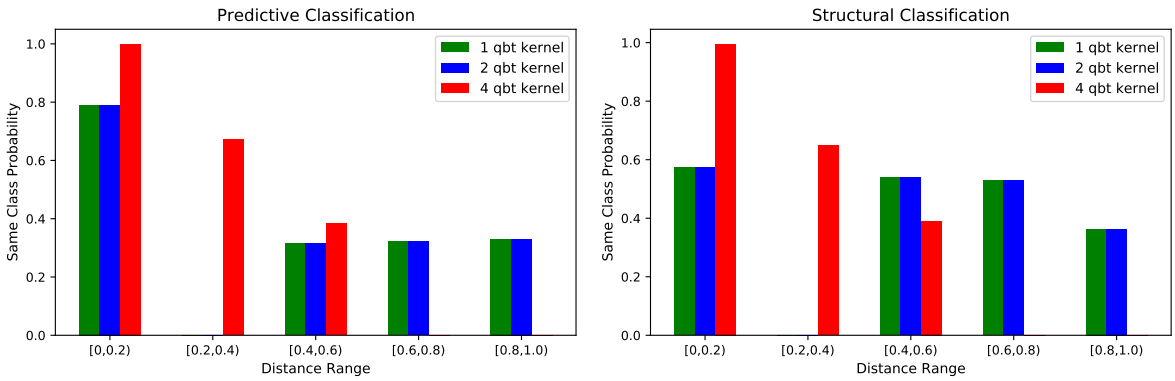

Figure 8: Probability of Examples Belonging to the Same Class - Structural Kernel, Bures Metric

6 Quantum Kernels in Quantum Circuits Computing Model

For each sequence $\mathbf{a} \in \Sigma^*$ a quantum generative model Q (18) defines a probability $P\big[\mathbf{a}|G\big]$, and hence G defines distribution on all continuations $\mathbf{b}, |\mathbf{b}| = t$ of the sequence:

$$D_t^{\mathbf{a}} = \big\{P\big[\mathbf{b}|\mathbf{a}, G\big] = P\big[\mathbf{ab}|G\big] : \mathbf{b} \in \Sigma^t\big\}$$

In many cases the class of a sequence depends on the current state and the probability of its continuations. For example, in a financial time series of price movements the class of a sequence can be 1 if the next movement is expected to be "UP". In an English sentence the class of a phrase is "subject" if the expected next word is a verb. In these cases we classify the sequence by the distributions of their continuations. Therefore it is natural to assume that if the induced future distributions of two sequences are close, the similarity of these sequences is high. Let's formalize this intuitive explanation. Let $\mathbf{a} = a_1 \dots a_k$ and $\mathbf{b} = b_1 \dots b_l$ are the sequences we want to compare. The model Q defines the following end states for each of them:

$$\rho_{\mathbf{a}} = T_{a_k} \dots T_{a_1} \rho_0$$
$$\rho_{\mathbf{b}} = T_{b_l} \dots T_{b_1} \rho_0$$

.

The use of quantum computing for non-linear mapping into a higher-dimensional vector space for non-linear classification was discussed for almost a decade in different sources, e.g. [53, 54, 55, 56, 57].

In the field of quantum machine learning a quantum kernel is often defined as the inner product between two data-encoding feature vectors $\rho_{\mathbf{a}}$, $\rho_{\mathbf{b}}$ [58]:

$$k_{1,2} = Tr\{\rho_{\mathbf{a}} \rho_{\mathbf{b}}\}$$

Classifier	Kernel	In Sample	CI	Out Sample	CI
RFS	N/A	0.998	0.996 - 0.999	0.968	0.966 - 0.971
SVC	Classical	0.951	0.947 - 0.955	0.916	0.910 - 0.922
SVC	Quantum	0.999	0.999 - 1.000	0.980	0.977 - 0.981
k-NN	Classical	0.971	0.969 - 0.973	0.916	0.913 - 0.919
k-NN	Quantum	0.993	0.992 - 0.994	0.980	0.978 - 0.982

Table 2: Kernels Performance by Recognizer Accuracy: Predictive Task, Predictive Kernel, Distance Metric- Trace, CI=95%, Kernel Type RBF

Classifier	Kernel	In Sample	CI	Out Sample	CI
RFS	N/A	0.985	0.984 - 0.987	0.824	0.819 - 0.829
SVC	Classical	0.962	0.959 - 0.965	0.903	0.896 - 0.909
SVC	Quantum	1.000	-	0.949	0.946 - 0.952
k-NN	Classical	0.863	0.857 - 0.869	0.720	0.715 - 0.723
k-NN	Quantum	0.950	0.948 - 0.953	0.893	0.889 - 0.897

Table 3: Kernels Performance by Recognizer Accuracy: Structural Task, Structural Kernel, Distance Metric- Trace, CI=95%, Kernel Type RBF

This definition works well, when feature maps are implemented by unitary operations on quantum circuits. In quantum computing there is a vital need to verify and characterize quantum states, where the fidelity is an important and useful similarity measure. There are a number of proposals for estimating the mixed state fidelity on a quantum computer, which overcome hardness limitations of classical algorithms. One proposal is a variational quantum algorithm for low-rank fidelity estimation [59]. Another proposal with exponential speedup [60] relies on state purifications being supplied by an oracle. Other variational algorithms have been proposed for the fidelity and trace distance [61]. A number of quantum measures of distinguishability are discussed in [62]. Special consideration should be given to working with mixed states. Classical fidelity measure was introduced by Richard Jozsa [63]. [64] gives most recent review of mixed states fidelity measures.

In the case of more general feature maps producing mixed states we will use definition 3 and equation (37).

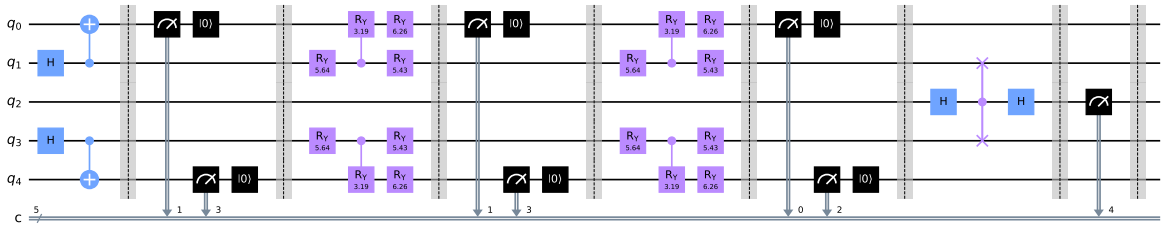

Figure 9: Implementation of SWAP test for market QHMM

In this section we will construct a kernel matrix for all pairs of sequences. One possible approach to implement this on the hardware is to use SWAP test [65] as shown on FIG 9. Qubits q_0, q_1 and q_3, q_4 on FIG 9 implement two QHMMs, q_2 is used for fidelity calculation. Maximally mixed states are prepared for both QHMMs leveraging classical registers 1 and 3. Then the circuit implements two steps of QHMM generator. Classical registers 1 and 0 capture the symbols of the first sequence, registers 3 and 2 of the second. Finally, the probability of measuring 0 on qubit q_2 captured on classical register 4 can be converted to the squared fidelity [65, 66] as

$$|\langle\phi|\psi\rangle|^2 = 2 \times (-0.5 + r_0 \times R^{-1})$$

where ϕ and ψ are state vectors of respective state systems of both QHMMs, r_0 is the number of observed zero bit-strings on qubit q_2 and R is the number of shots. The number of shots required is usually estimated as $O(n)$, where n is the number of states. In our case, $n = 2^{2t+1}$, where t is the length of the sequence. Unfortunately, for sequences of length 7-8 this requires about 1 million shots

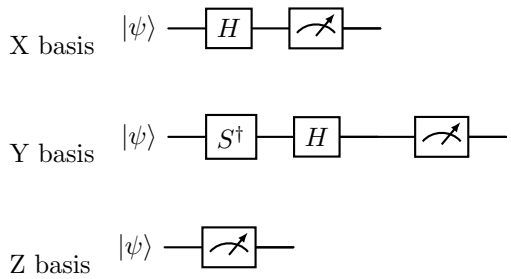

Figure 10: Measurements of the single qubit state in X, Y and Z bases

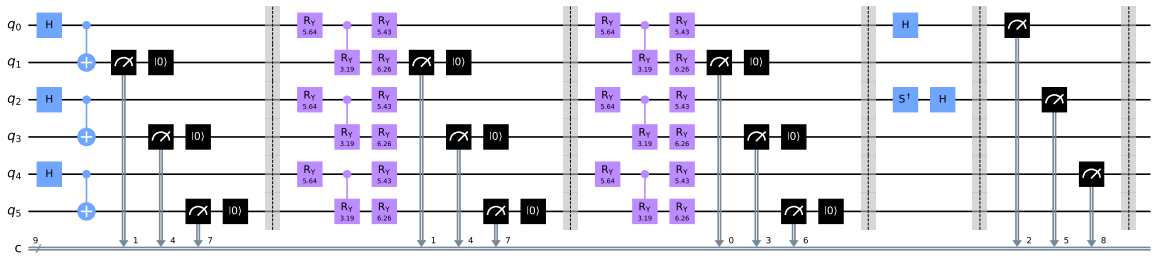

Figure 11: Implementation of projected kernel for market QHMM circuit

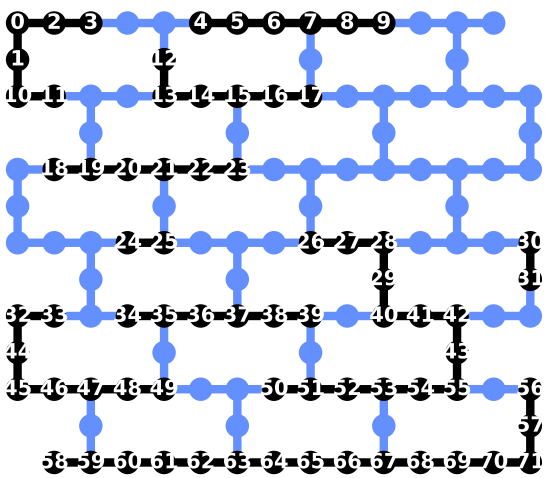

Figure 12: Initial qubit layout on *ibm_nazca* device for projected kernel approach

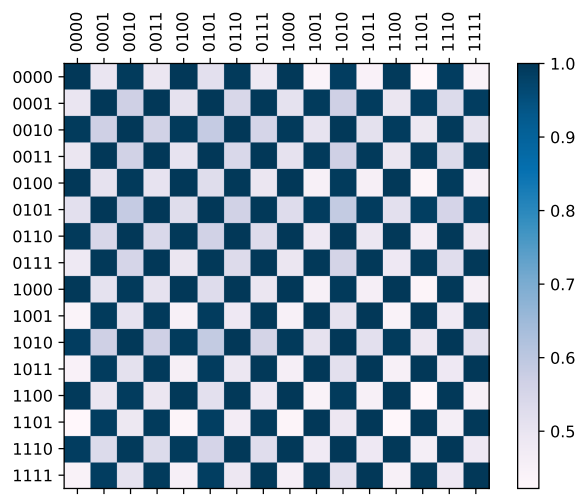

Figure 13: Color bar plot of the kernel matrix estimated on (*ibm_nazca*) device using projected kernels approach

or more. Another downside is that this approach assumes calculating kernel element for pure states. In general, we can be working with mixed states. A better approach is to use projected kernels [67]. It is well-known [50, 68] that a density matrix of a single qubit can be written as

$$\rho = \frac{1}{2}\left[1 + \sum_k r_k \sigma_k\right]$$

where σ_k are the Pauli matrices and r_k is a real, three component vector. ρ can be reconstructed from the three measurements results

$$r_k = \langle \sigma_k \rangle = tr(\rho \sigma_k)$$

.

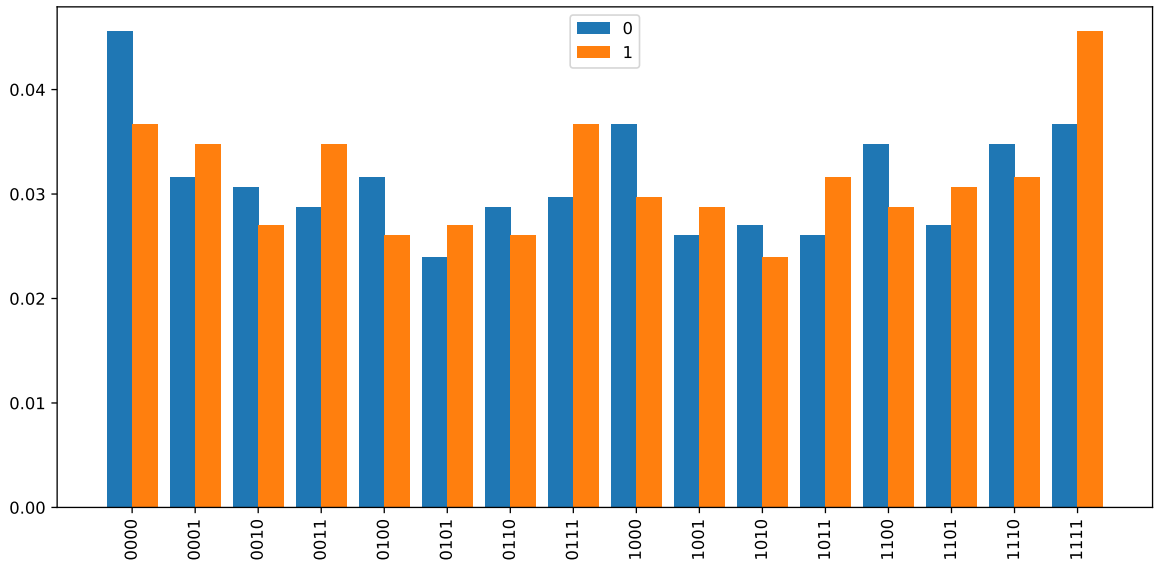

Figure 14: Next Symbol Distributions for 4-symbol Sequences

The circuit implementation [68] is shown on FIG 10. We can implement projected kernel for the market QHMM circuit as shown on FIG 11. Finally, the kernel element is constructed from two density matrices as $k_{1,2} = e^{-\gamma\|\rho_1-\rho_2\|^2}$, where $\|.\|$ is a Frobenius norm and γ is a hyperparameter (for now, we set $\gamma = 1$). In order to boost the effective number of shots we propose to run multiple circuits simultaneously in parallel on a single chip by combining them on a single circuit (multi-programming [69, 70, 71]). With this in mind, the initial qubit layout on *IBM Nazca* device is shown on FIG 12, where we effectively use 72 qubits. Color bar plot of the kernel matrix is shown on FIG 13. We see a perfect separation between classes. In this case the main system required only 1 qubit. In case, we have systems of many qubits we can use 1-reduced density matrix (1-RDM) or N-RDM approximations.

The checkerboard pattern is not surprising. Let's plot 5-symbol sequences on a histogram and split counts for the fifth symbol between two bars, one for 0 and one for 1 (see FIG 14). This way we can see, which is the most likely suffix for 4-symbol prefix. 0 and 1 alternate as we go from 0000 to 1111.

7 Conclusions and Outlook

In this article, we introduced a generative quantum computing approach to designing similarity measures and associated kernels for the classification of stochastic symbolic time series. The proposed kernels are built upon a novel class of quantum generative models known as quantum hidden Markov models. We provided theoretical justifications of the efficacy of the kernels in predictive classification tasks. Extensive simulation experiments have demonstrated the scalable and discriminative performance of the kernels in high-dimensional Hilbert spaces. Furthermore, we have devised and implemented the kernels in the quantum circuits computing model and successfully conducted experiments on quantum hardware. Finally, we compared the performance of the introduced quantum kernels to their classical counterparts on simple predictive and structural classification tasks, where the quantum kernels showed clear superiority. The proposed approach introduces novel quantum generative modeling techniques for designing hybrid classical-quantum algorithms for clustering, classification, and regression tasks in the domain of symbol sequences and time series. An active research direction involves applying the proposed kernels to the classification of high-frequency time series in the field of quantitative finance. Success in this challenging domain will require extending the underlying quantum stochastic model to encompass the generation of fractional and other non-Markovian stochastic processes.

Acknowledgements

The authors thank Amol Deshmukh for fruitful discussions about this work.

The views expressed in this article are those of the authors and do not represent the views of Wells Fargo. This article is for informational purposes only. Nothing contained in this article should be construed as investment advice. Wells Fargo makes no express or implied warranties and expressly disclaims all legal, tax, and accounting implications related to this article.

References

[1] H. Shimodaira, K.-i. Noma, M. Nakai, and S. Sagayama, "Dynamic time-alignment kernel in support vector machine," in *Advances in Neural Information Processing Systems* (T. Dietterich, S. Becker, and Z. Ghahramani, eds.), vol. 14, MIT Press, 2001.

[2] Y. Zheng, Q. Liu, E. Chen, Y. Ge, and J. L. Zhao, "Time series classification using multi-channels deep convolutional neural networks," in *Web-Age Information Management* (F. Li, G. Li, S.-w. Hwang, B. Yao, and Z. Zhang, eds.), (Cham), pp. 298–310, Springer International Publishing, 2014.

[3] J. Ye, C. Xiao, R. M. Esteves, and C. Rong, "Time series similarity evaluation based on spearman's correlation coefficients and distance measures," in *Cloud Computing and Big Data* (W. Qiang, X. Zheng, and C.-H. Hsu, eds.), (Cham), pp. 319–331, Springer International Publishing, 2015.

[4] S. Aghabozorgi, A. Seyed Shirkhorshidi, and T. Ying Wah, "Time-series clustering – a decade review," *Information Systems*, vol. 53, pp. 16–38, 2015.

[5] H. Shimodaira, K.-i. Noma, M. Nakai, and S. Sagayama, "Dynamic time-alignment kernel in support vector machine," in *Advances in Neural Information Processing Systems* (T. Dietterich, S. Becker, and Z. Ghahramani, eds.), vol. 14, MIT Press, 2001.

[6] L. Chen and R. Ng, "- on the marriage of lp-norms and edit distance," in *Proceedings 2004 VLDB Conference* (M. A. Nascimento, M. T. Özsu, D. Kossmann, R. J. Miller, J. A. Blakeley, and B. Schiefer, eds.), pp. 792–803, St Louis: Morgan Kaufmann, 2004.

[7] A. Stefan, V. Athitsos, and G. Das, "The move-split-merge metric for time series," *IEEE Transactions on Knowledge and Data Engineering*, vol. 25, no. 6, pp. 1425–1438, 2013.

[8] H. Li, C. Guo, and W. Qiu, "Similarity measure based on piecewise linear approximation and derivative dynamic time warping for time series mining," *Expert Systems with Applications*, vol. 38, no. 12, pp. 14732–14743, 2011.

[9] J. Lin, E. Keogh, S. Lonardi, and B. Chiu, "A symbolic representation of time series, with implications for streaming algorithms," in *Proceedings of the 8th ACM SIGMOD Workshop on Research Issues in Data Mining and Knowledge Discovery, DMKD '03*, pp. 2–11, 06 2003.

[10] J. Lines and A. Bagnall, "Time series classification with ensembles of elastic distance measures," *Data Mining and Knowledge Discovery*, vol. 29, pp. 565–592, 2015.

[11] L. Ye and E. Keogh, "Time series shapelets: a novel technique that allows accurate, interpretable and fast classification," *Data Mining and Knowledge Discovery*, vol. 22, pp. 149–182, 2011.

[12] B. Schölkopf and A. J. Smola, *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond*. MIT press Cambridge, 2001.

[13] J. Mercer, "Functions of positive and negativetypeand their connection with theory ofintegral equations," *Philosophical Trinsoctions of Royal Society*, pp. 4–415, 1909.

[14] V. N. Vapnik, *Statistical Learning Theory*. John Wiley & Sons, Inc, 1998.

[15] B. Boser, I. Guyon, and V. Vapnik, "A training algorithm for optimal margin classifiers," in *Proceedings of the Fifth Annual Workshop on Computational Learning Theory*, (Pittsburgh), 1992.

[16] K. P. Murphy, *Machine Learning: A Probabilistic Perspective.* The MIT Press, 2012.

[17] E. Pękalska and B. Haasdonk, "Kernel discriminant analysis for positive definite and indefinite kernels," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 31, no. 6, pp. 1017–1032, 2009.

[18] B. Schölkopf, A. Smola, and K. Müller, "Nonlinear component analysis as a kernel eigen-value problem," *Neural Computation*, vol. 10, no. 5, pp. 1299–1319, 1998.

[19] A. Pandey, H. De Meulemeester, B. De Moor, and J. A. Suykens, "Multi-view kernel pca for time series forecasting," *Neurocomputing*, vol. 554, p. 126639, 2023.

[20] B. Schölkopf, S. Mika, C. J. C. Burges, P. Knirsch, K. R. Müller, G. Raetsch, and A. Smola, "Input space vs. feature space in kernel-based methods," *IEEE Transactions on Neural Networks*, vol. 10, no. 5, pp. 1000–1017, 1999.

[21] F. R. Bach and M. I. Jordan, "Kernel independent component analysis," *Journal of Machine Learning Research*, vol. 3, pp. 1–48, 2002.

[22] A. Schell and H. Oberhauser, "Nonlinear independent component analysis for discrete-time and continuous-time signals," 2023.

[23] J. Shawe-Taylor and N. Cristianini, *Kernel methods for pattern analysis.* Cambridge university press, 2004.

[24] R. Langone, R. Mall, C. Alzate, and J. A. Suykens, "Kernel spectral clustering and applications," *Unsupervised learning algorithms*, pp. 135–161, 2016.

[25] A. Harvey and V. Oryshchenko, "Kernel density estimation for time series data," *International Journal of Forecasting*, vol. 28, no. 1, pp. 3–14, 2012. Special Section 1: The Predictability of Financial Markets Special Section 2: Credit Risk Modelling and Forecasting.

[26] H. Lei and B. Sun, "A study on the dynamic time warping in kernel machines," in *2007 Third International IEEE Conference on Signal-Image Technologies and Internet-Based System*, pp. 839–845, 2007.

[27] M. Badiane and P. Cunningham, "An empirical evaluation of kernels for time series," *Artif Intell Rev*, vol. 55, pp. 1803—1820, 2022.

[28] T. Smith and M. Waterman, "Identification of common molecular subsequences," *J. Mol. Biol.*, vol. 147, pp. 195–197, 1981.

[29] J.-P. Vert, H. Saigo, and T. Akutsu, "Local alignment kernels for biologicalsequences," in *Kernel Methods in Computational Biology* (B. Scholkopf, K. Tsuda, and J. Vert, eds.), p. 131–154, MIT Press, 2004.

[30] D. Haussler, "Convolution kernels on discrete structures," Technical Report UCSC-CRL-99-10, UC Santa Cruz, 1999.

[31] D. Haussler, "Convolution kernels on discrete structures," Tech. Rep. UCSC-CRL-99-10, University of California in Santa Cruz, Computer Science Department, July 1999.

[32] T. Jebara, R. Kondor, and A. G. Howard, "Probability product kernels," *J. Mach. Learn. Res.*, vol. 5, pp. 819–844, 2004.

[33] P. Moreno, P. Ho, and N. Vasconcelos, "A kullback-leibler divergence based kernel for svm classification in multimedia applications," in *Advances in Neural Information Processing Systems* (S. Thrun, L. Saul, and B. Schölkopf, eds.), vol. 16, MIT Press, 2003.

[34] M. Cuturi, K. Fukumizu, and J.-P. Vert, "Semigroup kernels on measures," *Journal of Machine Learning Research*, vol. 6, no. 40, pp. 1169–1198, 2005.

[35] T. Jaakkola, M. Diekhans, and D. Haussler, "Using the fisher kernel method to detect remote protein homologies," in *Proceedings of the International Conference on Intelligent Systems for Molecular Biology*, August 1999.

[36] T. Jaakkola and D. Haussler, "Exploiting generative models in discriminative classifiers," in *Advances in Neural Information Processing Systems*, pp. 487–493, 1999.

[37] V. Markov, V. Rastunkov, A. Deshmukh, D. Fry, and C. Stefanski, "Implementation and learning of quantum hidden markov models," 2023.

[38] H. GM, M. K. Gourisaria, M. Pandey, and S. S. Rautaray, "A comprehensive survey and analysis of generative models in machine learning," *Computer Science Review*, vol. 38, p. 100285, 2020.

[39] H. Jaeger, M. Zhao, K. Kretzschmar, T. Oberstein, D. Popovici, and A. Kolling, "Learning observable operator models via the es algorithm," *New directions in statistical signal processing: From systems to brains*, 2005.

[40] J. Carlyle and A. Paz, "Realizations by stochastic finite automata," *Journal of Computer and System Sciences*, vol. 5, no. 1, pp. 26–40, 1971.

[41] R. Dhanuka, J. P. Singh, and A. Tripathi, "A comprehensive survey of deep learning techniques in protein function prediction," *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, vol. 20, no. 3, pp. 2291–2301, 2023.

[42] J. Jurgovsky, M. Granitzer, K. Ziegler, S. Calabretto, P.-E. Portier, L. He-Guelton, and O. Caelen, "Sequence classification for credit-card fraud detection," *Expert Syst. Appl.*, vol. 100, pp. 234–245, 2018.

[43] A. A. A. Odaini, F. Zola, L. Segurola-Gil, A. Gil-Lertxundi, and C. D'Andrea, "Cybersecurity in public space: Leveraging cnn and lstm for proactive multivariate time series classification," in *2023 IEEE International Conference on Big Data (BigData)*, pp. 4110–4118, 2023.

[44] S. Sharma, K. K. Bhatt, R. Chabra, and N. Aneja, "A comparative performance model of machine learning classifiers on time series prediction for weather forecasting," in *Advances in Information Communication Technology and Computing* (V. Goar, M. Kuri, R. Kumar, and T. Senjyu, eds.), (Singapore), pp. 577–587, Springer Nature Singapore, 2022.

[45] M. Liang, X. Wang, and S. Wu, "Improving stock trend prediction through financial time series classification and temporal correlation analysis based on aligning change point," *Soft Comput*, vol. 27, p. 3655–3672, 2023.

[46] L. Rosafalco, A. Manzoni, and S. Mariani, "Fully convolutional networks for structural health monitoring through multivariate time series classification," *Adv. Model. and Simul. in Eng. Sci.*, vol. 7, p. 3655–3672, 2020.

[47] N. Tsalikidis, A. Mystakidis, and C. Tjortjis, "Energy load forecasting: one-step ahead hybrid model utilizing ensembling," *Computing*, vol. 106, p. 241–273, 2024.

[48] B. Gomes, J. Coelho, and H. Aidos, "A survey on traffic flow prediction and classification," *Intelligent Systems with Applications*, vol. 20, p. 200268, 2023.

[49] A. Monras, A. Beige, and K. Wiesner, "Hidden quantum markov models and non-adaptive read-out of many-body states," *Applied Mathematical and Computational Sciences*, vol. 3, no. 1, pp. 93–122, 2011.

[50] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information: 10th Anniversary Edition.* Cambridge University Press, 2010.

[51] W. F. Stinespring, "Positive functions on C*-algebras," *Proceedings of the American Mathematical Society*, vol. 6, no. 2, p. 211–216, 1955.

[52] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[53] P. Rebentrost, M. Mohseni, and S. Lloyd, "Quantum support vector machine for big data classification," *Phys. Rev. Lett.*, vol. 113, p. 130503, Sep 2014.

[54] R. Chatterjee and T. Yu, "Generalized coherent states, reproducing kernels, and quantum support vector machines," *Quantum Information and Computation*, vol. 17, Dec. 2017.

[55] M. Schuld and N. Killoran, "Quantum machine learning in feature hilbert spaces," *Physical review letters*, vol. 122, no. 4, p. 040504, 2019.

[56] J.-E. Park, B. Quanz, S. Wood, H. Higgins, and R. Harishankar, "Practical application improvement to quantum svm: theory to practice," 2020.

[57] V. Rastunkov, J.-E. Park, A. Mitra, B. Quanz, S. Wood, C. Codella, H. Higgins, and J. Broz, "Boosting method for automated feature space discovery in supervised quantum machine learning models," 2022.

[58] M. Schuld and F. Petruccione, *Machine learning with quantum computers*. Springer, 2021.

[59] M. Cerezo, A. Poremba, L. Cincio, and P. J. Coles, "Variational quantum fidelity estimation," *Quantum*, vol. 4, p. 248, 2020.

[60] Q. Wang, Z. Zhang, K. Chen, J. Guan, W. Fang, J. Liu, and M. Ying, "Quantum algorithm for fidelity estimation," *IEEE Transactions on Information Theory*, vol. 69, no. 1, pp. 273–282, 2022.

[61] R. Chen, Z. Song, X. Zhao, and X. Wang, "Variational quantum algorithms for trace distance and fidelity estimation," *Quantum Science and Technology*, vol. 7, no. 1, p. 015019, 2021.

[62] C. A. Fuchs, "Distinguishability and accessible information in quantum theory," 1996.

[63] R. Jozsa, "Fidelity for mixed quantum states," *Journal of modern optics*, vol. 41, no. 12, pp. 2315–2323, 1994.

[64] Y.-C. Liang, Y.-H. Yeh, P. E. Mendonça, R. Y. Teh, M. D. Reid, and P. D. Drummond, "Quantum fidelity measures for mixed states," *Reports on Progress in Physics*, vol. 82, no. 7, p. 076001, 2019.

[65] H. Buhrman, R. Cleve, J. Watrous, and R. de Wolf, "Quantum fingerprinting," *Phys. Rev. Lett.*, vol. 87, p. 167902, Sep 2001.

[66] V. Havlíček, A. D. Córcoles, K. Temme, A. W. Harrow, A. Kandala, J. M. Chow, and J. M. Gambetta, "Supervised learning with quantum-enhanced feature spaces," *Nature*, vol. 567, no. 7747, pp. 209–212, 2019.

[67] H.-Y. Huang, M. Broughton, M. Mohseni, R. Babbush, S. Boixo, H. Neven, and J. R. McClean, "Power of data in quantum machine learning," *Nature Communications*, vol. 12, no. 2631, 2021.

[68] A. J., A. Adedoyin, J. Ambrosiano, P. Anisimov, W. Casper, G. Chennupati, C. Coffrin, H. Djidjev, D. Gunter, S. Karra, N. Lemons, S. Lin, A. Malyzhenkov, D. Mascarenas, S. Mniszewski, B. Nadiga, D. O'malley, D. Oyen, S. Pakin, L. Prasad, R. Roberts, P. Romero, N. Santhi, N. Sinitsyn, P. J. Swart, J. G. Wendelberger, B. Yoon, R. Zamora, W. Zhu, S. Eidenbenz, A. Bärtschi, P. J. Coles, M. Vuffray, and A. Y. Lokhov, "Quantum algorithm implementations for beginners," *ACM Transactions on Quantum Computing*, vol. 3, pp. 1–92, jul 2022.

[69] P. Das, S. S. Tannu, P. J. Nair, and M. Qureshi, "A case for multi-programming quantum computers," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, pp. 291–303, 2019.

[70] L. Liu and X. Dou, "Qucloud: A new qubit mapping mechanism for multi-programming quantum computing in cloud environment," in *2021 IEEE International symposium on high-performance computer architecture (HPCA)*, pp. 167–178, IEEE, 2021.

[71] S. Niu and A. Todri-Sanial, "Multi-programming mechanism on near-term quantum computing," in *Quantum Computing: Circuits, Systems, Automation and Applications*, pp. 19–54, Springer, 2023.

A Market Quantum Generative Models in Higher Dimensions

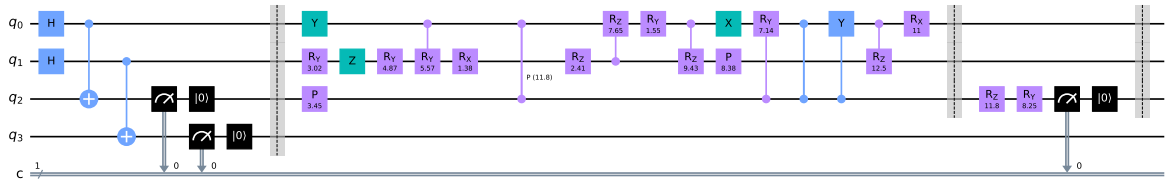

Figure 15: Market Movements QHMM: Two State Qubits; One Emission Qubit

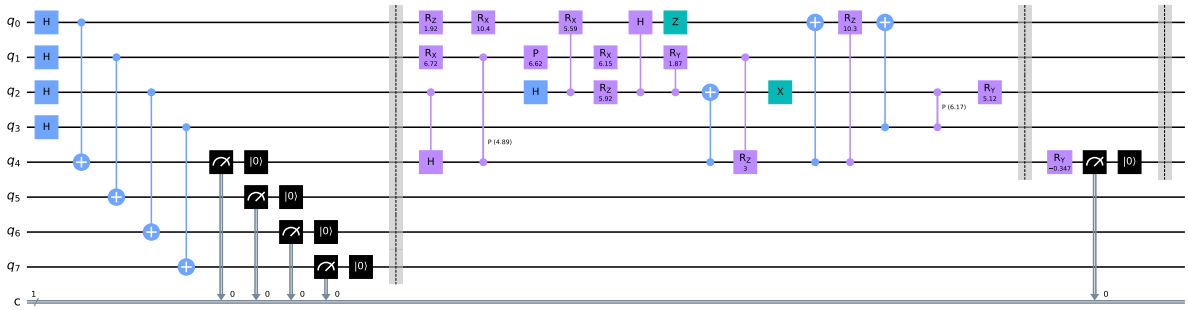

Figure 16: Market Movements QHMM: 4 State Qubits; One Emission Qubit

B Classification Tasks

We empirically study binary classification problem for binary time series in the domain for market movement process (Example 1). The generative model of the example domain is a QHMM (26) parameterized in Hilbert spaces with dimensions 2, 4 and 8 (Figures 3, 15, 16).
A structural classificataion task is defined by the following class-mapping function (11):

$$p_s(\mathbf{y}, c) = \begin{cases} 1 & \text{if } c = 1 \wedge \sum_{i=1}^{|\mathbf{y}|} y_i > \frac{|\mathbf{y}|}{2} \\ 0 & \text{if } c = 0 \wedge \sum_{i=1}^{|\mathbf{y}|} y_i \leq \frac{|\mathbf{y}|}{2} \end{cases} \tag{51}$$

The predictive classification task uses class-mapping function (12) defined for forward sequences with length $k = 5$:

$$p_p(\mathbf{y}, k, c) = L[c_2],$$

where c_2 is the binary number corresponding to the prefix and the label of each prefix is defined by

$$L =' 11110011001100100110001011000110'$$

The examples $\mathbf{y} \in \{0, 1\}^*$ are sampled with probability (22):

$$P\big[\mathbf{y}|\rho_0\big] = \mathrm{tr}(T_{\mathbf{y}}\rho_0).$$

C Kernel Induced Distances Across Classification Tasks - Trace Metric

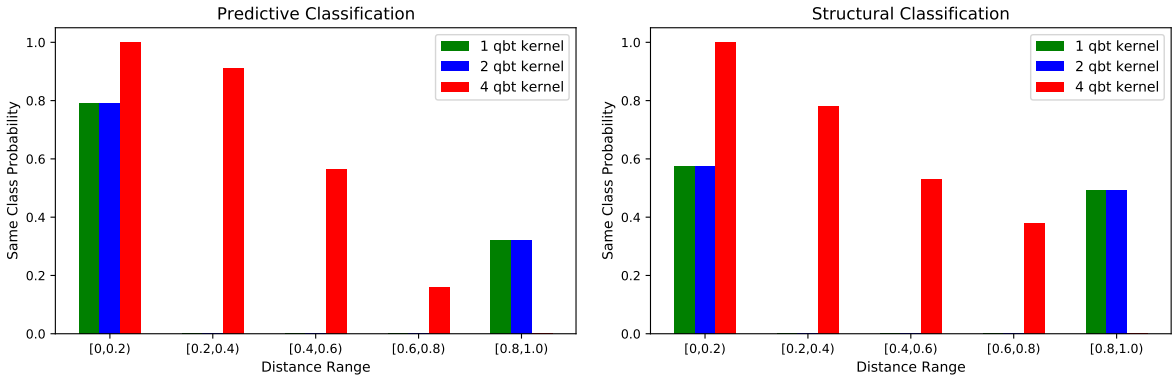

Figure 17: Probability of Examples Belonging to the Same Class -Predictive Kernel, Trace Metric

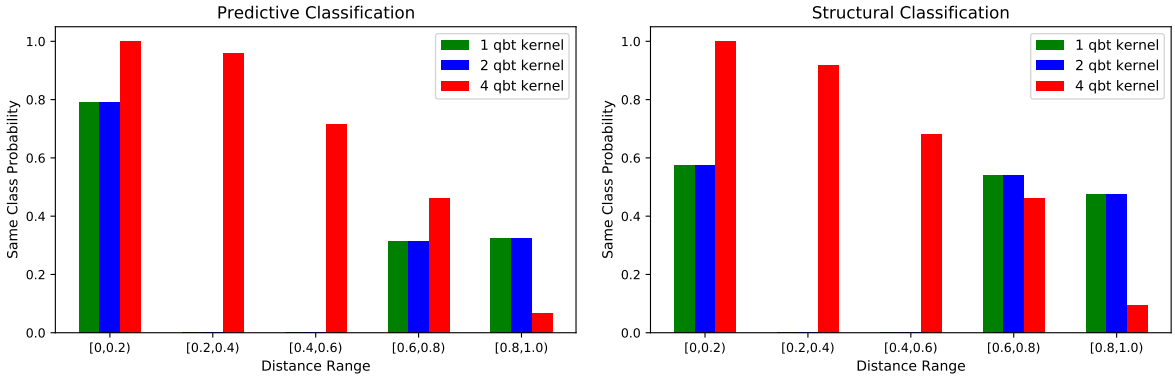

Figure 18: Probability of Examples Belonging to the Same Class - Structural Kernel, Trace Metric

D Kernels Performance - Distance Metric Bures

Classifier	Kernel	In Sample	CI	Out Sample	CI
RFS	N/A	0.985	0.984 - 0.987	0.824	0.819 - 0.829
SVC	Classical	0.962	0.959 - 0.965	0.903	0.896 - 0.909
SVC	Quantum	0.998	0.997 - 0.998	0.947	0.944 - 0.949
k-NN	Classical	0.863	0.857 - 0.869	0.720	0.715 - 0.723
k-NN	Quantum	0.953	0.951 - 0.956	0.894	0.891 - 0.898

Table 4: Kernels Performance by Recognizer Accuracy: Structural Task, Structural Kernel, Distance Metric- Bures, CI=95%, Kernel Type RBF

Classifier	Kernel	In Sample	CI	Out Sample	CI
RFS	N/A	0.998	0.996 - 0.999	0.968	0.966 - 0.971
SVC	Classical	0.951	0.947 - 0.955	0.916	0.910 - 0.922
SVC	Quantum	0.999	0.998 - 0.999	0.977	0.975 - 0.979
k-NN	Classical	0.971	0.969 - 0.973	0.916	0.913 - 0.919
k-NN	Quantum	0.994	0.993 - 0.995	0.979	0.977 - 0.981

Table 5: Kernels Performance by Recognizer Accuracy: Predictive Task, Predictive Kernel, Distance Metric- Bures, CI=95%, Kernel Type RBF